

05. Práctica del tidyverse

Ciencia de datos para la toma de decisiones I

Jorge
Juvenal
Campos Ferreira

✉ juvenal.campos@tec.mx

Estructura (sugerida) de un código o script

1. Configuración
2. Carga de librerías
3. Carga de parámetros y funciones propias
4. Carga de datos
5. Limpieza de datos
6. Análisis exploratorio de datos
7. Generación de funciones
8. Generación y pruebas de modelos
9. Generación de gráficas
10. Guardado de archivos

```
# -----  
# Autor: Juvenal Campos  
# Fecha: 2025-08-18  
# Proyecto: Ejemplo para el Tec de Monterrey  
# Descripción: Script dummy que ilustra la estructura básica de un flujo en R.  
# -----  
  
# 1) Configuración -----  
options(digits = 3)  
set.seed(42)  
  
# 2) Carga de librerías -----  
library(dplyr)  
library(readr)  
library(ggplot2)  
  
# 3) Parámetros y funciones propias -----  
params <- list(  
  input = "data/input.csv",      # <- reemplaza si tienes archivo  
  output_dir = "out",  
  target = "mpg"  
)  
paleta_colores <- c("#6950d8", "#3CEAFA", "#00b783",  
                    "#ff6260", "#ffaf84", "#ffbd41")  
if (!dir.exists(params$output_dir)) dir.create(params$output_dir, recursive = TRUE)  
  
clean_names <- function(x){  
  names(x) <- tolower(gsub("[^a-z0-9_]+", "_", names(x)))  
  x  
}  
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))  
  
# 4) Carga de datos -----  
# df <- read_csv(params$input)      # <- real  
# Dummy para ilustrar:  
df <- clean_names(mtcars)
```

Estructura (sugerida) de un código o script

1. Configuración
2. Carga de librerías
3. Carga de parámetros y funciones propias
4. Carga de datos
- 5. Limpieza de datos**
- 6. Análisis exploratorio de datos**
- 7. Generación de funciones**
- 8. Generación y pruebas de modelos**
- 9. Generación de gráficas**
- 10. Guardado de archivos**

```
# 5) Limpieza de datos -----
df <- df %>%
  mutate(across(everything(), \(v) v)) %>%
  filter(!is.na(.data[[params$target]]))

# 6) Análisis exploratorio (EDA) -----
print(summary(df))
print(df %>% summarise(n = n(), p = ncol(.)))

# 7) Generación de funciones -----
# (ya definimos `rmse()` arriba como ejemplo)

# 8) Modelado (dummy) -----
mod <- lm(mpg ~ wt + hp, data = df)
pred <- predict(mod, df)
cat("RMSE:", rmse(df$mpg, pred), "\n")

# 9) Gráficas -----
p <- ggplot(df, aes(wt, mpg)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)

ggsave(file.path(params$output_dir, "scatter.png"), p,
        width = 5, height = 4, dpi = 150)

# 10) Guardado de archivos -----
write_csv(df, file.path(params$output_dir, "data_clean.csv"))
saveRDS(mod, file.path(params$output_dir, "model.rds"))
cat("Archivos guardados en:", normalizePath(params$output_dir), "\n")
```

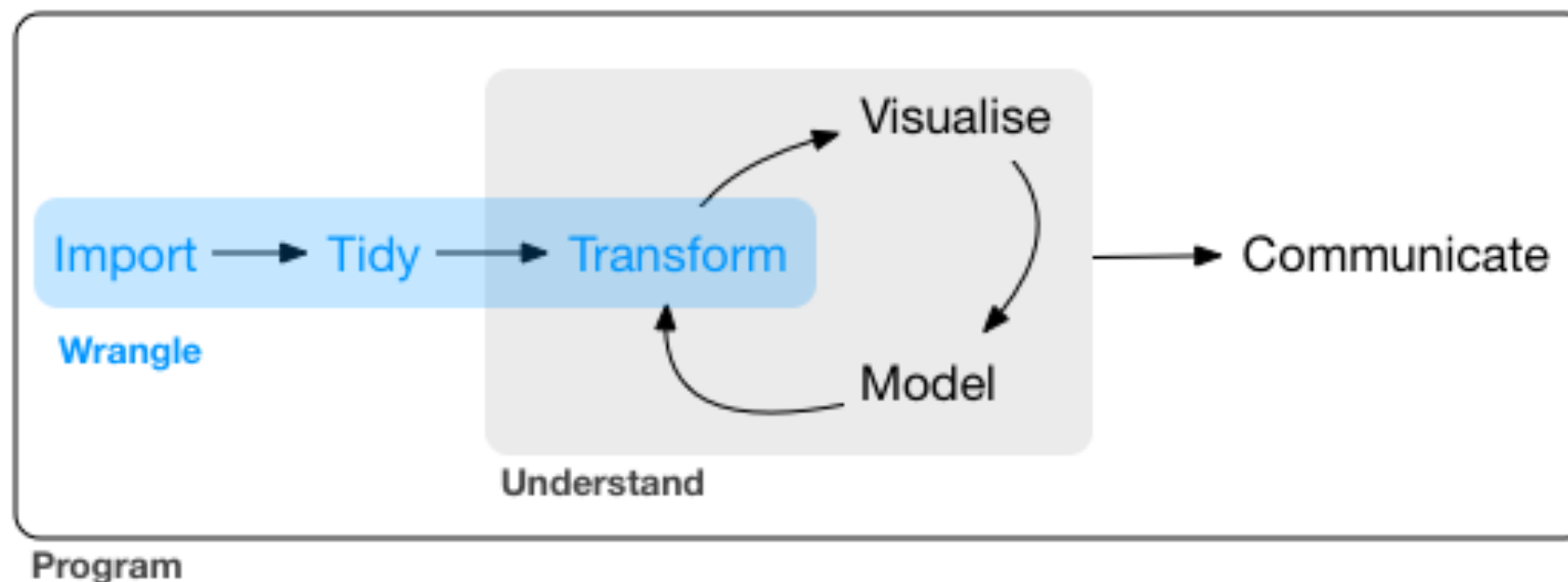


Procesamiento de datos

@allison_horst

Procesamiento de datos

- * Es el **proceso previo** a la creación de análisis, modelos y visualizaciones.
- * Consiste en **tomar una base de datos** y darles una estructura que nos sea útil para generar análisis, gráficas o modelos.
- * Siempre hay que tener clara a qué estructura queremos llegar.



Procesamiento de datos

Ejemplo de algunos procesos:

- 1) Limpieza de datos
 - 1) Convertir tipos de datos (texto -> numero)
 - 2) Eliminar registros con datos faltantes (NAs)
 - 3) Eliminar duplicados y outliers
 - 4) Estandarizar categorías (“CdMx” -> “CDMX”)
- 2) Unir tablas
- 3) Reestructurar tablas
- 4) Agregar datos por grupos
- 5) Cambiar nombres de columnas
- 6) Reclasificar grupos
- 7) Deflactar cifras
- 8) Transformar valores
- 9) Reconvertir columnas

¿Qué es el tidyverse?

El **tidyverse** es una colección de paquetes de R diseñados para su uso en ciencia de datos. Todos los paquetes miembros comparten una filosofía de diseño, estructura de datos y gramática comunes.



@allison_horst

Paquetes del tidyverse



Familia tidyverse (y este amiguito -> %>%)

Paquetes del tidyverse



- La pipa es el elemento central del tidyverso y su misión es la de **concatenar funciones**. Opera vinculando los objetos del lado izquierdo con las funciones del lado derecho.
- Para escribirlo se utiliza:
 - **MAC**: command + shift + m
 - **Windows**: control + shift + m

Operador pipa (%>%)



De manera coloquial, el operador pipa se debe leer como un **"y después"**. Por ejemplo, tomemos el objeto Juve.

* Con pipa:

```
# Un objeto Juve
```

```
Juve %>%
```

```
  despertarse(hora = "5:00 a.m.") %>% # Juve despierto (y despues...,
```

```
  levantarse(hora = "5:10 a.m.") %>% # Juve levantado (y despues...)
```

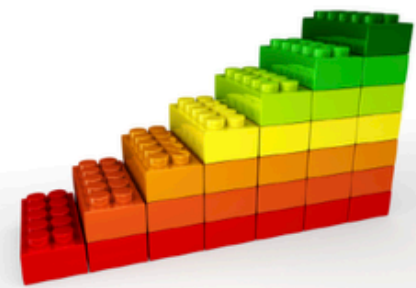
```
  bañarse() %>% # Juve despierto, levantado y bañado (y despues...)
```

```
  vestirse() %>% # Juve ahira vestido (y despues...)
```

```
  desayunar() %>% # Juve ahora desayunado (y despues...)
```

```
  salir_al_trabajo() # Juve rumbo al CIDE
```

Legos



```
# Sin la pipa
```

```
salir_al_trabajo(desayunar(vestirse(bañarse(levantarse(despertarse(Juve, hora = "5:00 a.m."), hora = "5:10 a.m.")))))
```

```
# Reasignando sobre el mismo objeto
```

```
Juve <- despertarse(Juve, hora = "5:00 a.m.")
```

```
Juve <- levantarse(Juve, hora = "5:10 a.m.")
```

```
Juve <- bañarse(Juve)
```

```
Juve <- vestirse(Juve)
```

```
Juve <- desayunar(Juve)
```

```
Juve <- salir_al_trabajo(Juve)
```

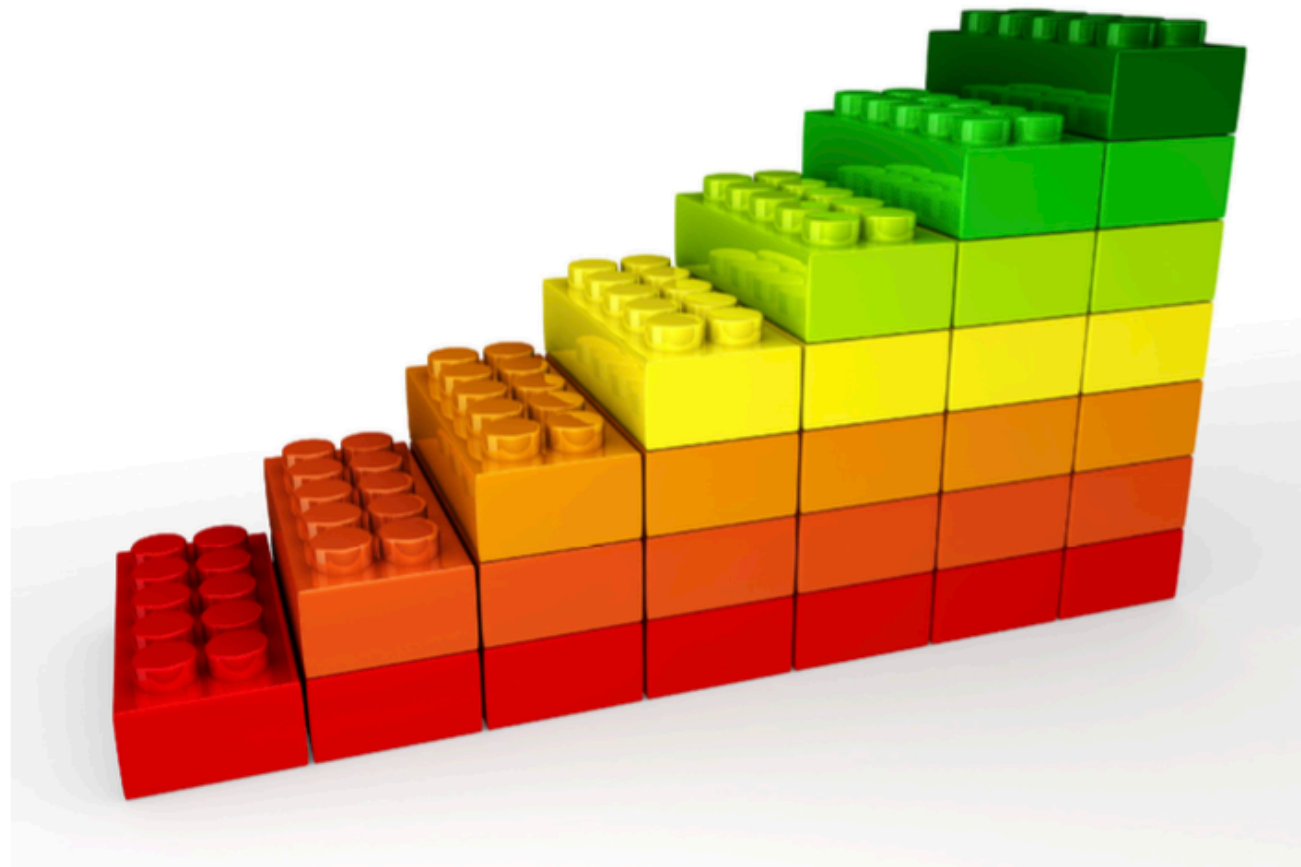
Capas de cebolla



Operador pipa (%>%)



El operador %>% toma un objeto (en este caso Juve) y le va aplicando funciones (en este caso, todas las relativas a su rutina diaria matutina) de manera secuencial. Esto se va haciendo a manera de la unión de bloques de lego para armar una torre.



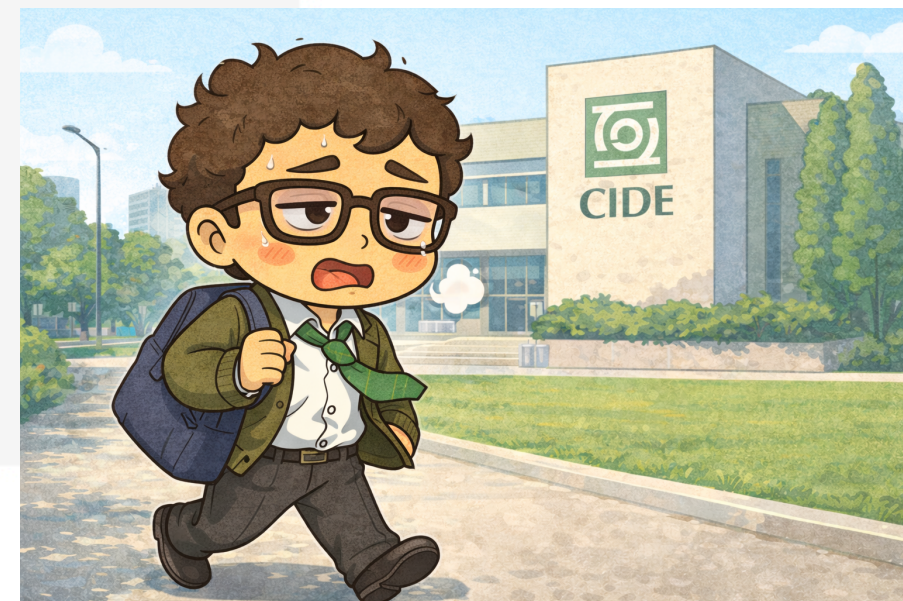
Operador pipa (%>%)



Tip 1 Si un día te sale un error y no sabes en que parte del pipeline se encuentra, corre una por una los bloques de código hasta que encuentres el error.

Tip 2 La estructura de trabajo con pipelines nos permite quitar (o silenciar, convirtiendo código en comentario) pedazos de código de manera muy fácil para hacer pruebas. Por ejemplo:

```
# Un objeto Juve
Juve %>%
  despertarse(hora = "8:00 a.m.") %>% # Juve despierto
  levantarse(hora = "8:10 a.m.") %>% # Juve levantado
#   bañarse() %>% # Juve despierto, levantado y bañado
  vestirse() %>% # Juve vestido
#   desayunar() %>% # Juve desayunado
  salir_al_trabajo() # Juve rumbo al CIDE
```



Operador pipa (%>%)



* Sin pipa:

```
# Declaramos el vector numérico `x`  
x <- c(0.109, 0.359, 0.63, 0.996, 0.515, 0.142, 0.017, 0.829, 0.907)
```

```
# Obtenemos el logaritmo de X, sacamos diferencias de un periodo, al resultado le  
sacamos la exponencial y después redondeamos el resultado.  
round(exp(diff(log(x))), 1)
```

```
## [1] 3.3 1.8 1.6 0.5 0.3 0.1 48.8 1.1
```

Capas de cebolla



```
x <- log(x)  
x <- diff(x)  
x <- exp(x)  
x <- round(x, 1)  
x
```

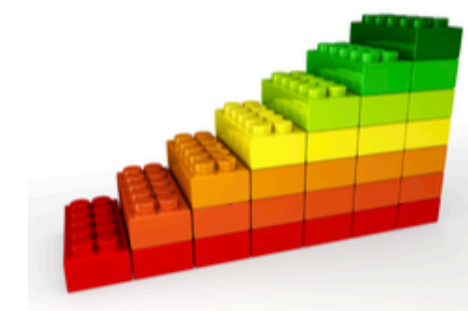
* Con pipa:

```
# La pipa ya viene importada en el tidyverse
```

```
# Hacemos las mismas operaciones que arriba:  
x %>% log() %>%  
diff() %>%  
exp() %>%  
round(1)
```

```
## [1] 3.3 1.8 1.6 0.5 0.3 0.1 48.8 1.1
```

Legos



Verbos del tidyverse



Las principales funciones del **tidyverse** para tratar con datos son las siguientes:

- **filter()**, para filtrar renglones,
- **select()**, para seleccionar columnas,
- **mutate()**, para generar nuevas columnas,
- **arrange()**, para ordenar los renglones en función de una variable,
- **group_by()**, para generar grupos o clusters de renglones,
- **summarise()**, para generar cálculos con las agrupaciones del **group_by()**

Analogía: los verbos del tidyverse

*Piensa en dplyr como un lenguaje con verbos. Así como en español usas verbos para describir acciones (filtrar, seleccionar, crear, ordenar, resumir), en dplyr cada acción sobre los datos tiene un verbo específico: **filter()**, **select()**, **mutate()**, **arrange()**, **summarise()**. Si sabes qué quieres hacer con tus datos, solo necesitas elegir el verbo correcto.*



filter()

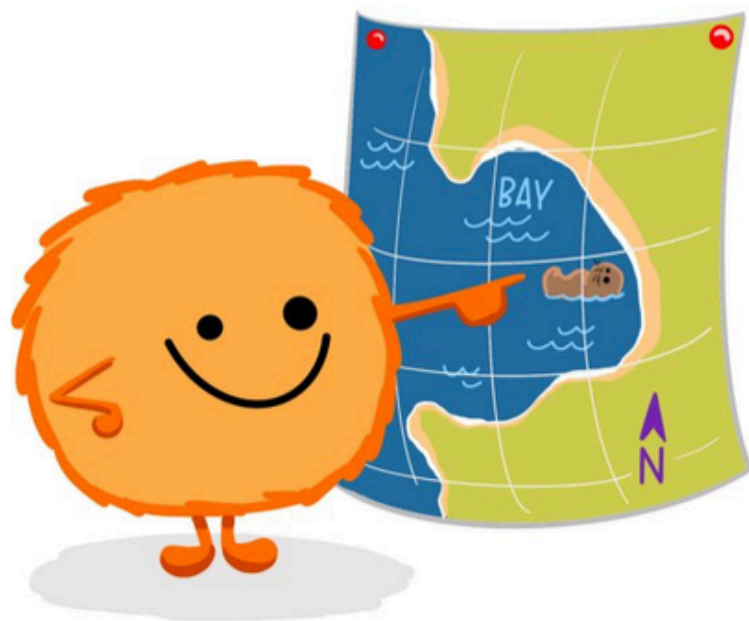


dplyr::filter()

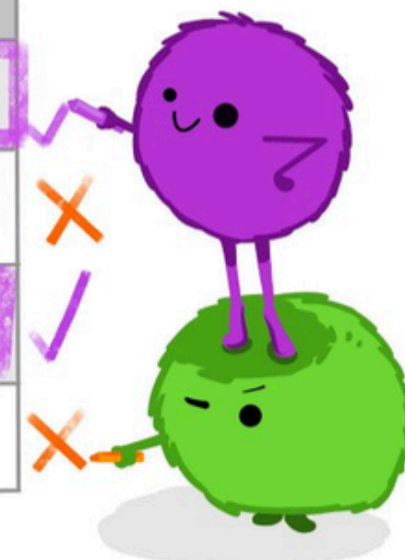
KEEP ROWS THAT
satisfy
your CONDITIONS

keep rows from... this data... ONLY IF... type is "otter" AND site is "bay"

```
filter(df, type == "otter" & site == "bay")
```



type	food	site
otter	urchin	bay
shark	seal	channel
otter	abalone	bay
otter	crab	wharf



@allison-horst

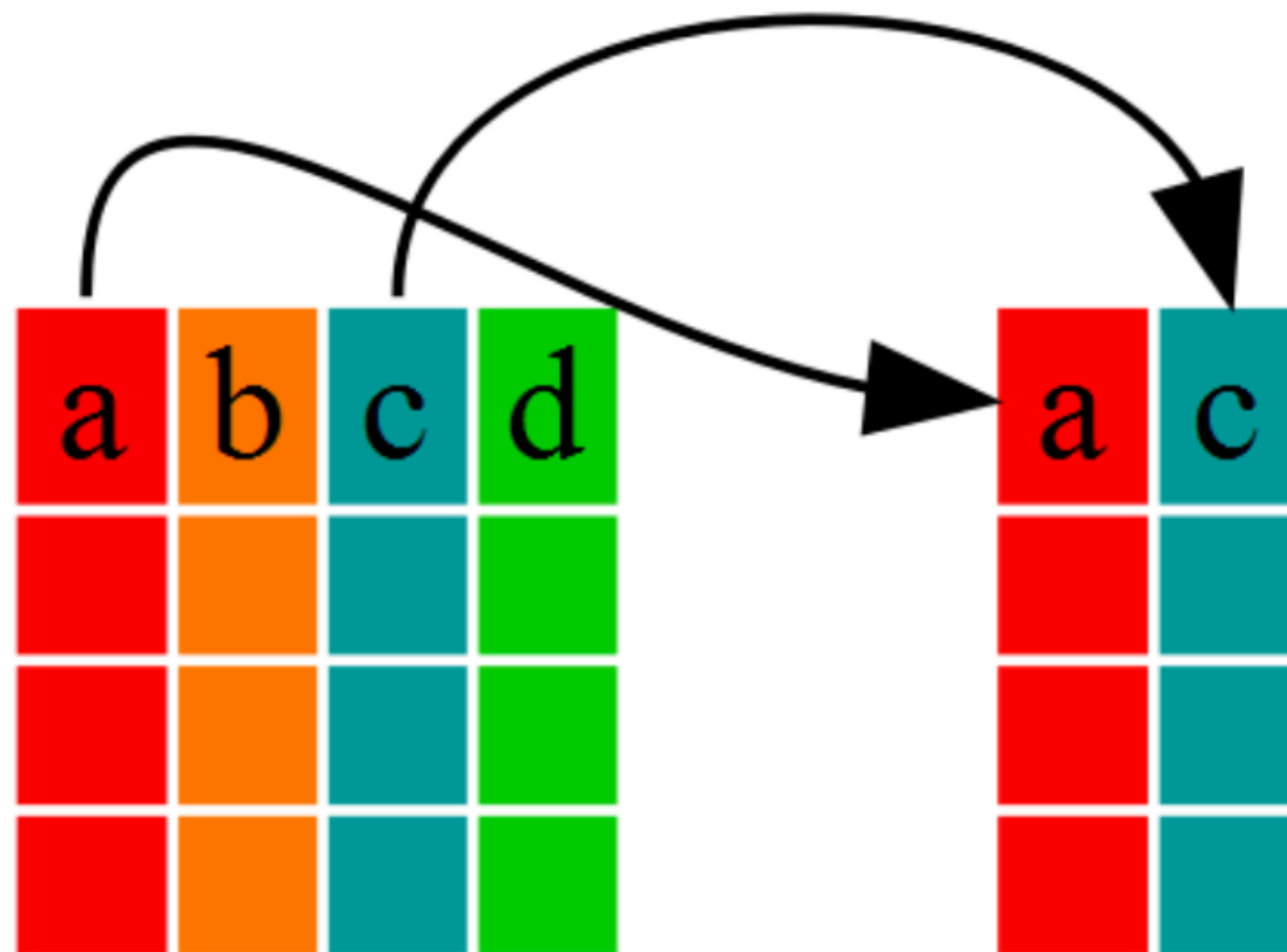
Operadores

Operador/Función	Significado	Ejemplo
<code>==</code>	Igual a	<code>region == "Sur"</code>
<code>!=</code>	Diferente de	<code>estado != "Oaxaca"</code>
<code>> < >= <=</code>	Comparaciones numéricas	<code>pobreza > 40</code>
<code>& o ,</code>	Y (ambas condiciones)	<code>pobreza > 40, region == "Sur"</code>
<code> </code>	O (alguna condición)	<code>region == "Sur" region == "Norte"</code>
<code>!</code>	Negación (NOT)	<code>!es_urbano</code>
<code>%in%</code>	Pertenece a un conjunto	<code>estado %in% c("Oaxaca","Chiapas")</code>
<code>between()</code>	En un rango	<code>between(pobreza, 20, 40)</code>
<code>is.na()</code>	Es valor faltante	<code>is.na(ingreso)</code>

select()



```
select(data.frame,a,c)
```



mutate()



arrange()



arrange()

storms

storm	wind	pressure	date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1999-06-13
Arthur	45	1010	1996-06-21



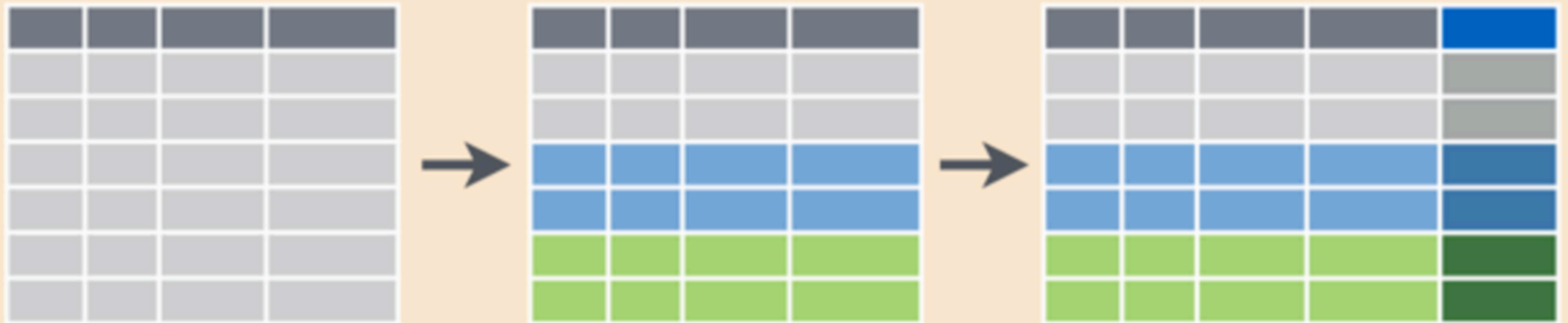
storm	wind	pressure	date
Ana	40	1013	1997-07-01
Alex	45	1009	1998-07-30
Arthur	45	1010	1996-06-21
Arlene	50	1010	1999-06-13
Allison	65	1005	1995-06-04
Alberto	110	1007	2000-08-12

group_by()



Group Data

Compute new variables by group.



Intuición: group_by() como dividir una baraja de cartas

Imagina que tienes una baraja de cartas (tu data frame). **group_by()** es como separar las cartas por palo (corazones, diamantes, tréboles, picas). **summarise()** es como contar cuántas cartas hay en cada montón, cuál es el valor promedio, etc. Sin **group_by()**, **summarise()** opera sobre toda la baraja junta.



summarise()



city	particle size	amount ($\mu\text{g}/\text{m}^3$)
New York	large	23
New York	small	14



city	mean	sum	n
New York	18.5	37	2

London	large	22
London	small	16



London	19.0	38	2
--------	------	----	---

Beijing	large	121
Beijing	small	56



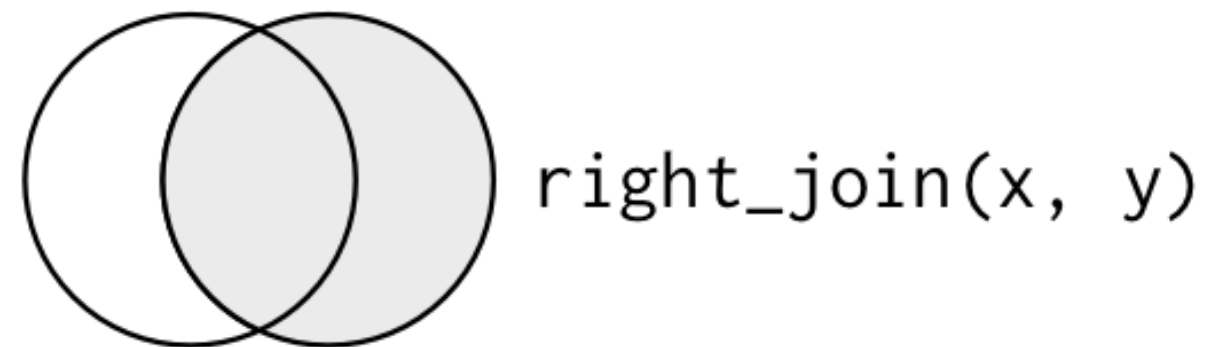
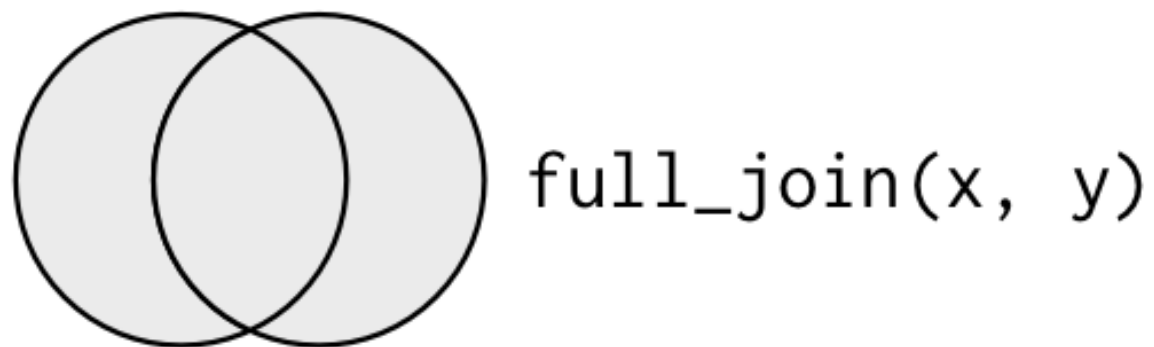
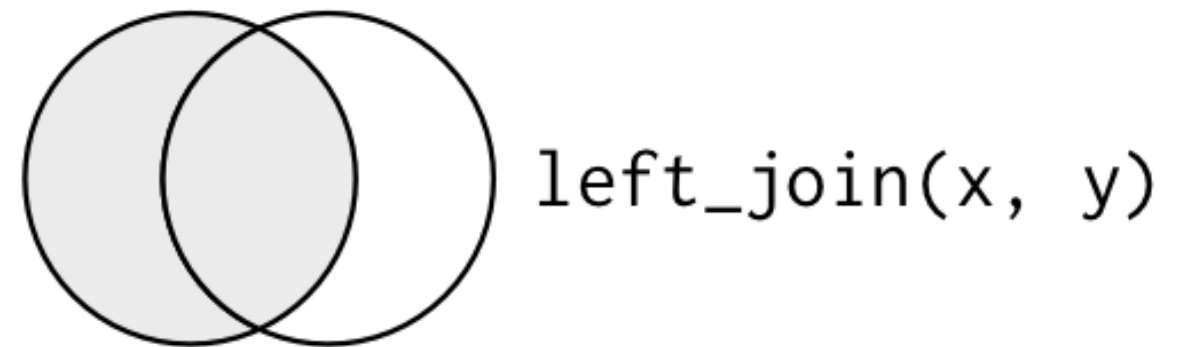
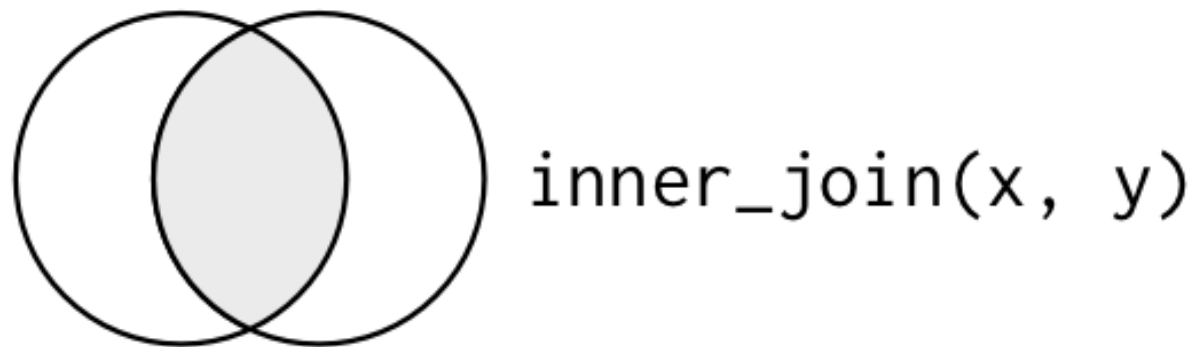
Beijing	88.5	177	2
---------	------	-----	---

Funciones útiles dentro de un summarise

Función	Qué calcula	Ejemplo
n()	Número de filas	n()
mean()	Promedio	mean(ingreso, na.rm = TRUE)
median()	Mediana	median(escolaridad)
sd()	Desviación estándar	sd(pobreza)
sum()	Suma total	sum(poblacion)
min(), max()	Mínimo y máximo	min(pobreza), max(pobreza)
first(), last()	Primer y último valor	first(nombre)
n_distinct()	Valores únicos	n_distinct(estado)

Uniando tablas (joins)

En el mundo real, la información está dispersa en múltiples tablas. Los joins (uniones) permiten combinar dos data frames usando una columna común como llave (key):



Revisar: https://rpubs.com/Juve_Campos/juntando_tablas



Relaciones entre tablas

Relación 1 a 1, cuando un valor de una tabla hace match con el valor de otra tabla.

Relación 1 a muchos, cuando un valor de una tabla se corresponde a muchos valores de la otra tabla.

Relación muchos a muchos, cuando varios valores de una tabla se relacionan con varios otros valores de otras tablas. Esta situación es rara y no es muy deseable, en general.

Ejemplo

casos

Municipios	Casos
Ocuituco	10
Yecapixtla	13
Jonacatepec	23
Tetela	5
Tepoztlán	50

fall

Municipios	Fallecimientos
Yecapixtla	1
Jonacatepec	2
Tepoztlán	20

hosp

Municipios	Hospitales
Ocuituco	1
Jonacatepec	1
Tetela	1
Tepoztlán	2

pop

Municipios	Poblacion
Ocuituco	18000
Yecapixtla	40000
Jonacatepec	35000
Tetela	20000
Tepoztlán	45000

left_join()

`left_join(casos, fall, by = "Municipios")`

Municipios	Casos	Fallecimientos
Ocuituco	10	NA
Yecapixtla	13	1
Jonacatepec	23	2
Tetela	5	NA
Tepoztlán	50	20

`left_join(fall, hosp, by = "Municipios")`

Municipios	Fallecimientos	Hospitales
Yecapixtla	1	NA
Jonacatepec	2	1
Tepoztlán	20	2

`left_join(casos, pop, by = "Municipios")`

Municipios	Casos	Poblacion
Ocuituco	10	18000
Yecapixtla	13	40000
Jonacatepec	23	35000
Tetela	5	20000
Tepoztlán	50	45000

Ejemplo

casos

Municipios	Casos
Ocuituco	10
Yecapixtla	13
Jonacatepec	23
Tetela	5
Tepoztlán	50

fall

Municipios	Fallecimientos
Yecapixtla	1
Jonacatepec	2
Tepoztlán	20

hosp

Municipios	Hospitales
Ocuituco	1
Jonacatepec	1
Tetela	1
Tepoztlán	2

pop

Municipios	Poblacion
Ocuituco	18000
Yecapixtla	40000
Jonacatepec	35000
Tetela	20000
Tepoztlán	45000

`inner_join()`

`inner_join(casos, fall, by = "Municipios")`

Municipios	Casos	Fallecimientos
Yecapixtla	13	1
Jonacatepec	23	2
Tepoztlán	50	20

`inner_join(fall, hosp, by = "Municipios")`

Municipios	Fallecimientos	Hospitales
Jonacatepec	2	1
Tepoztlán	20	2

`inner_join(casos, pop, by = "Municipios")`

Municipios	Casos	Poblacion
Ocuituco	10	18000
Yecapixtla	13	40000
Jonacatepec	23	35000
Tetela	5	20000
Tepoztlán	50	45000

Ejemplo

casos

Municipios	Casos
Ocuituco	10
Yecapixtla	13
Jonacatepec	23
Tetela	5
Tepoztlán	50

fall

Municipios	Fallecimientos
Yecapixtla	1
Jonacatepec	2
Tepoztlán	20

hosp

Municipios	Hospitales
Ocuituco	1
Jonacatepec	1
Tetela	1
Tepoztlán	2

pop

Municipios	Poblacion
Ocuituco	18000
Yecapixtla	40000
Jonacatepec	35000
Tetela	20000
Tepoztlán	45000

full_join()

`full_join(casos, fall, by = "Municipios")`

Municipios	Casos	Fallecimientos
Ocuituco	10	NA
Yecapixtla	13	1
Jonacatepec	23	2
Tetela	5	NA
Tepoztlán	50	20

`full_join(fall, hosp, by = "Municipios")`

Municipios	Fallecimientos	Hospitales
Ocuituco	NA	1
Yecapixtla	1	NA
Jonacatepec	2	1
Tetela	NA	1
Tepoztlán	20	2

`full_join(casos, pop, by = "Municipios")`

Municipios	Casos	Poblacion
Ocuituco	10	18000
Yecapixtla	13	40000
Jonacatepec	23	35000
Tetela	5	20000
Tepoztlán	50	45000

Reestructurar datos

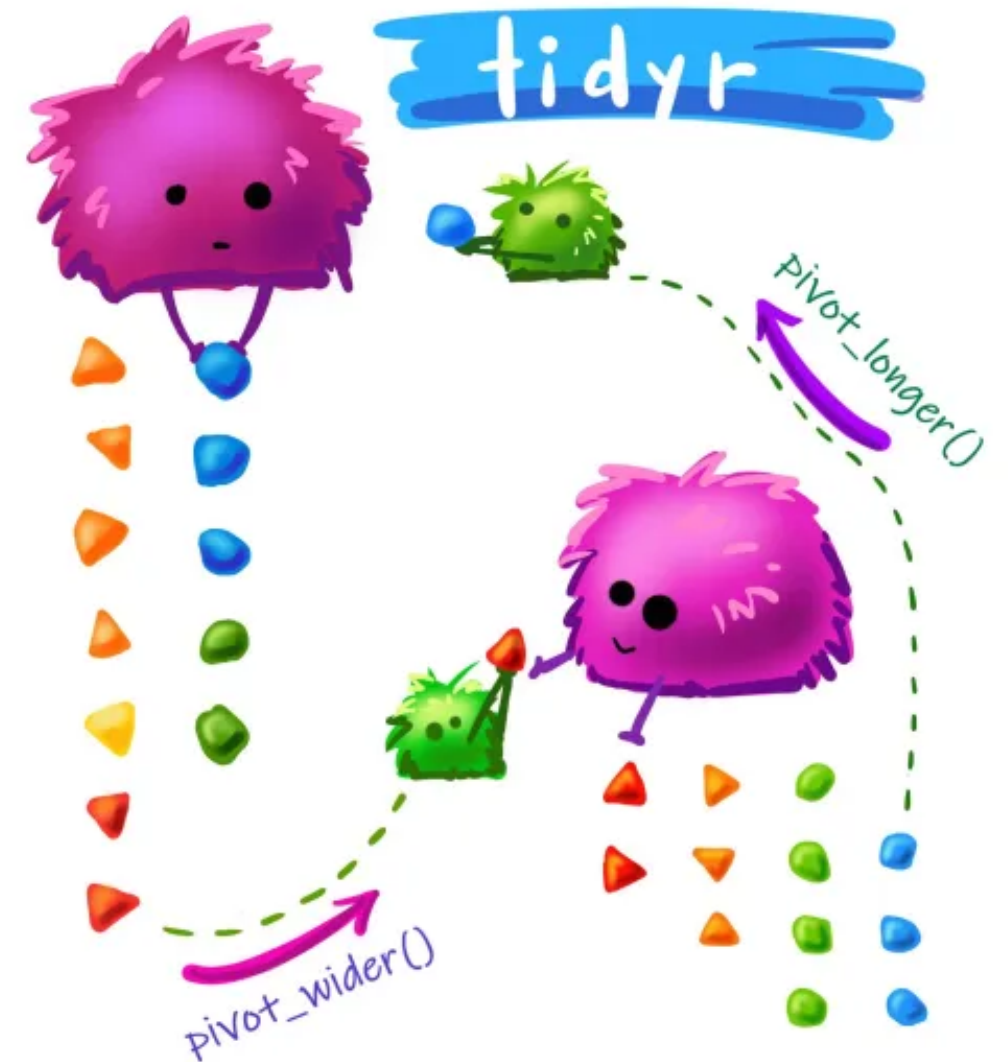
A veces los datos no llegan en el formato que necesitas. Las dos operaciones más comunes son pivotar entre formato largo y formato ancho:

wide

id	x	y	z
1	a	c	e
2	b	d	f

long

id	key	val
1	x	a
2	x	b
1	y	c
2	y	d
1	z	e
2	z	f



Revisar: <https://juvenalcampos.com/posts/2020-12-04-pivoteando-bases/>



pivot_longer()

country	1999	2000	2001	2002
Angola	800	750	925	1020
India	20100	25650	26800	27255
Mongolia	450	512	510	586

Pivot data longer

```
data %>%  
  pivot_longer(  
    cols = 1999:2002,  
    names_to = "year",  
    values_to = "cases"  
  )
```



country	year	cases
Angola	1999	800
Angola	2000	750
Angola	2001	925
Angola	2002	1020
India	1999	20100
India	2000	25650
India	2001	26800
India	2002	27255
Mongolia	1999	450
Mongolia	2000	512
Mongolia	2001	510
Mongolia	2002	586

pivot_wider()

country	year	metric
x	1960	10
x	1970	13
x	2010	15
y	1960	20
y	1970	23
y	2010	25
z	1960	30
z	1970	33
z	2010	35

`pivot_wider(names_from = "year",
names_prefix = "yr",
values_from = "metric")`

country	yr1960	yr1970	yr2010
x	10	13	15
y	20	23	25
z	30	33	35

`pivot_longer(cols = yr1960:yr2010,
names_to = "year",
names_prefix = "yr",
values_to = "metric")`

Ejercicio de procesamiento de datos

- 1. Cargue los datos de pobreza que se encuentran en su carpeta del ejercicio bajo el nombre ***df_pob_ent2.xlsx***.
- 2. Utilizando los verbos del tidyverse vistos, responda las siguientes preguntas:
 - ¿Cuántas columnas tiene? ¿Cuántas filas?
 - ¿Qué representa cada fila en la tabla?
 - ¿Cuál es el estado con mayor porcentaje de población de pobreza (pobreza) en 2024?
 - ¿Cuál es el estado con menor porcentaje de carencia por acceso a la salud (ic_asalud) en 2024?

Ejercicio de procesamiento de datos

- ¿Cuales son los 10 estados con mayor porcentaje de pobreza extrema en 2024 (pobreza_e)?
- ¿Cuál es el promedio simple del porcentaje de pobreza moderada de los estados del centro del país (CDMX, Puebla, EDOMEX, Morelos e Hidalgo) en 2024?
- ¿En qué año alcanzó Chiapas su porcentaje más alto de población con carencia por acceso a la seguridad social?
- ¿Cuál es el valor más bajo de pobreza que ha alcanzado algún estado (con los datos de la tabla)?
- Obtenga el promedio y la desviación estándar del porcentaje de pobreza para todos los estados, para cada año

Ejercicio II

- En la carpeta del ejercicio 2, tiene una tabla de población, una tabla de pobreza municipal y una tabla de grado promedio de educación.
- Imagine que la Secretaría del Bienestar le pide priorizar los municipios que recibirán un nuevo programa de transferencias, y le pide seleccionar municipios con pobreza $> 30\%$, escolaridad promedio < 9 años y priorizando los más poblados. **Detecte cuales serían los 10 municipios prioritarios.**

Ejercicio III

- En la carpeta de ejercicio 3, cargue los datos provenientes de INEGI (PIB trimestral) SIN modificar la fuente primaria.
- Pase la tabla a formato largo
- ¿Cuál es el sector que más ha crecido desde el **2009T4**?